UNIVERSIDAD
Panamericana

PROGRAMACIÓN AVANZADA

Sesión 17

Taller 1 — Resolución de problemas

✎ NOTAS DEL PROFESOR

Universidad Panamericana
Dr. Miguel Alcaraz Rivera • malcaraz@up.edu.mx

2026

Índice

1 Objetivos de la sesión **2**

2 Dinámica del taller **2**

3 Problema 1: Paradoja de los promedios **4**
3.1 Enunciado completo 4
3.2 Parte 1 — Guía de pizarrón 4
3.3 Análisis de la solución 4
3.4 Verificación con la muestra 5
3.5 Solución en C++ 5

4 Problema 2: Ayudando a mi hermanito **7**
4.1 Enunciado 7
4.2 Parte 1 — Guía de pizarrón 7
4.3 Análisis de la solución 7
4.4 Solución en C++ 7

5 Materiales y recursos **9**

Objetivos de la sesión

Al finalizar la sesión el alumno será capaz de:

1. Leer un enunciado de concurso y extraer la condición matemática central.
2. Convertir una condición con promedios en aritmética entera (evitar flotantes innecesarios).
3. Implementar la reversión de un entero con operaciones de dígitos.
4. Enviar una solución a un juez en línea e interpretar el veredicto.

90 minutos. Distribución sugerida:
10 min — explicación de jueces en línea + dinámica del taller
8 min — **Problema 1, Parte 1:** revisión guiada en pizarrón
25 min — **Problema 1, Parte 2:** trabajo en equipo + envío al juez
8 min — **Problema 2, Parte 1:** revisión guiada en pizarrón
29 min — **Problema 2, Parte 2:** trabajo en equipo + envío al juez
10 min — cierre: discusión de soluciones y casos esquina

Los alumnos trabajan en equipos de dos o tres personas. Usan el cuaderno Jupyter (`sesiones/sesion17_taller_1.ipynb`) para probar localmente y envían la solución en C++ al juez al terminar.

Dinámica del taller

Preguntas de sondeo (5 min):

- "¿Alguien ha enviado código a un juez en línea antes?" — Esperar manos; si hay, pedirles que cuenten su experiencia.
- "Si saco a la persona con el promedio más bajo del salón y la mando a otro salón, ¿qué pasa con el promedio de este salón?" — Respuesta esperada: sube. "¿Y con el del otro?" — Depende de si está por debajo del promedio de ese salón. Este es exactamente el problema 1.

Los alumnos trabajan en **equipos de dos o tres personas**. Cada problema tiene dos momentos: primero la revisión guiada en el pizarrón (todos atentos, sin código), después el trabajo en equipo.

El objetivo de la Parte 1 **no** es enseñar la solución: es mostrar cómo leer un enunciado de concurso y trazar el caso de prueba a mano. Sigue este guión en el pizarrón:

1. **Lee el enunciado en voz alta** con los alumnos. Detente en cada párrafo y pregunta: "¿qué nos están dando?", "¿qué nos están pidiendo?".

2. **Escribe la entrada en el pizarrón** tal como aparece en el ejemplo, indicando qué representa cada número ("esta línea tiene $T = 1$ caso", "aquí $n = 5$ y $m = 5$", etc.).
3. **Traza el procesamiento a mano**, paso a paso, usando números concretos (sumas, promedios, comparaciones). Escribe cada operación, no la des por obvia.
4. **Escribe la salida** que se obtiene y verifica que coincide con la del enunciado.
5. Pregunta: "¿qué patrón general ven aquí?" — No respondas todavía; esa es la tarea del equipo en la Parte 2.

Asegúrate de que todos los alumnos tengan cuenta en omegaUp antes de la sesión. El problema 2 está en: `https://www.omegaup.com/arena/problem/La-tarea-de-mi-hermanito/`

Problema 1: Paradoja de los promedios

• Enunciado completo

Un chiste conocido dice que si un mal estudiante de Cómputo se cambia a Economía, sube el promedio de inteligencia en ambas facultades.

Dado el IQ de todos los estudiantes de dos facultades, determina cuántos estudiantes de Cómputo podrían protagonizar el chiste (es decir, cuántos harían que **ambos** promedios suban al cambiar de facultad).

Entrada: primera línea con T casos. Cada caso precedido por línea en blanco. Cada caso: primera línea con n y m (al menos 2 estudiantes de Cómputo); las siguientes líneas contienen $n + m$ enteros positivos (primeros n son Cómputo, los m restantes Economía).

Salida: un entero por caso.

Muestra: $n = 5$, $m = 5$, Cómputo: 100 101 102 103 104, Economía: 98 100 102 99 101. Salida: 1.

• Parte 1 — Guía de pizarrón

Escribe columna izquierda: la entrada. Columna derecha: las operaciones.

1. "Hay $T = 1$ caso de prueba."
2. "$n = 5$ estudiantes de Cómputo, $m = 5$ de Economía."
3. "IQs de Cómputo: 100, 101, 102, 103, 104. Suma $= 510$. Promedio $= 510/5 = 102$."
4. "IQs de Economía: 98, 100, 102, 99, 101. Suma $= 500$. Promedio $= 500/5 = 100$."
5. Para cada estudiante de Cómputo, escribe su IQ y evalúa:
   "¿Este IQ está *entre* 100 y 102?" (Señala 100, 101, 102, 103, 104 uno por uno.)
6. Solo el 101 cae estrictamente entre 100 y 102. "Por eso la respuesta es **1**."
7. Pregunta de cierre: "¿Qué pasa si el promedio de Cómputo fuera igual o menor al de Economía?" — No respondas; déjalo para el equipo.

• Análisis de la solución

Sea $\bar{c} = \text{sum_cs}/n$ el promedio de Cómputo y $\bar{e} = \text{sum_ec}/m$ el promedio de Economía.

Un estudiante con IQ x cumple el chiste si y solo si:

Condición 1 — sube el promedio de Cómputo al salir:

$$\frac{\text{sum_cs} - x}{n-1} > \frac{\text{sum_cs}}{n} \iff x \cdot n < \text{sum_cs} \iff x < \bar{c}$$

Condición 2 — sube el promedio de Economía al entrar:

$$\frac{\text{sum_ec} + x}{m+1} > \frac{\text{sum_ec}}{m} \iff x \cdot m > \text{sum_ec} \iff x > \bar{e}$$

Condición combinada: $\bar{e} < x < \bar{c}$.

Si $\bar{c} \leq \bar{e}$, ningún estudiante cumple (el rango está vacío). La verificación entera $x \cdot n <$ sum_cs y $x \cdot m >$ sum_ec evita comparar doubles. Con IQs hasta 10^4 y hasta 10^4 estudiantes, el producto cabe en `long long`.

• Verificación con la muestra

sum_cs $= 510$, $\bar{c} = 102$. sum_ec $= 500$, $\bar{e} = 100$.

x	$x \cdot 5 < 510$?	$x \cdot 5 > 500$?	¿Cuenta?
100	500 < 510 ✓	500 > 500 ×	No
101	505 < 510 ✓	505 > 500 ✓	**Sí**
102	510 < 510 ×	—	No
103	515 < 510 ×	—	No
104	520 < 510 ×	—	No

Resultado: **1**. ✓

• Solución en C++

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int T;
    cin >> T;
    while (T--) {
        int n, m;
        cin >> n >> m;
        vector<long long> cs(n), ec(m);
        for (auto& x : cs) cin >> x;
        for (auto& x : ec) cin >> x;

        long long sum_cs = 0, sum_ec = 0;
        for (auto x : cs) sum_cs += x;
        for (auto x : ec) sum_ec += x;

        int cnt = 0;
        for (auto x : cs)
            if (x * n < sum_cs && x * m > sum_ec)
                cnt++;
```

```
        cout << cnt << "\n";
    }
}
```

¿Por qué no basta con calcular los promedios como `double`?
Con IQs enteros y promedios exactos la diferencia puede ser 0.5 o menos. La comparación $x < \bar{c}$ con `double` puede dar falsos positivos o negativos por error de representación. Con enteros es exacto.

¿Qué pasa si $n = 2$ y el estudiante que sale es el único que lo haría?
No hay caso especial: la condición $x \cdot n <$ sum_cs sigue siendo válida para $n \geq 2$. Al salir quedan $n - 1 \geq 1$ estudiantes, que es un promedio bien definido.

¿Por qué `long long`?
Si los IQs llegan a 10^4 y n llega a 10^4, el producto $x \cdot n$ puede ser 10^8, que cabe en `int` (máx. $\approx 2 \times 10^9$), pero sum_cs con 10^4 elementos de valor 10^4 es 10^8, que también cabe. Usar `long long` es buena práctica defensiva aunque no sea estrictamente necesario aquí.

Problema 2: Ayudando a mi hermanito

• Enunciado

Se dan dos números de tres dígitos A y B sin ceros. Se invierten ambos y se devuelve el mayor de los dos *ya invertidos*, escrito en reversa.

Ejemplos:

A	B	reversa(A), reversa(B)	Salida
735	892	537, 298	537
331	341	133, 143	143

• Parte 1 — Guía de pizarrón

1. Escribe en el pizarrón: `735  892`. "Estos son los dos números que le doy a mi hermano."
2. "Él los lee al revés. ¿Cómo se lee 735 al revés?" — Esperar respuesta del salón: 537.
3. Escribe: $735 \to 537$ y $892 \to 298$.
4. "¿Cuál es mayor, 537 o 298?" — 537.
5. "¿Qué imprimimos? El mayor *leído en reversa* — es decir, 537, que ya está escrito al revés. Imprimimos 537."
6. Repite con el segundo ejemplo: $331 \to 133$, $341 \to 143$; mayor: 143; salida: 143.
7. Pregunta de cierre: "¿Cómo sacamos cada dígito de un número de tres dígitos usando código?" — No respondas; es la tarea de la Parte 2.

• Análisis de la solución

Para un número n de tres dígitos con dígitos $d_1 d_2 d_3$ (centenas, decenas, unidades):

$$\text{rev}(n) = (n \bmod 10) \times 100 + \Big((n/10) \bmod 10\Big) \times 10 + (n/100)$$

Como A y B no contienen ceros, la reversa de un número de tres dígitos siempre es también un número de tres dígitos (no hay ceros que desaparezcan al principio).

El problema es directo una vez que se identifica la fórmula de reversión. La restricción "sin ceros" elimina el único caso especial (cero inicial en la reversa que haría que la salida tuviera menos de tres dígitos). No se necesita `string`.

• Solución en C++

```
#include <iostream>
```

```
using namespace std;

int rev(int n) {
    return (n % 10) * 100 + ((n / 10) % 10) * 10 + n / 100;
}

int main() {
    int A, B;
    cin >> A >> B;
    int rA = rev(A), rB = rev(B);
    cout << (rA > rB ? rA : rB) << "\n";
}
```

¿Por qué no funciona con ceros?
Si $A = 300$, su reversa es 003, que como entero es 3 (dos dígitos menos). La salida sería 3 y no 003. El enunciado garantiza que esto no ocurre.

Un alumno usa `string` para invertir. ¿Es correcto?
Sí, también funciona. La solución con operaciones aritméticas es más eficiente y muestra mejor comprensión de la estructura de dígitos.

¿Es posible que las reversas sean iguales?
El enunciado dice que $A \neq B$. Pero podría ocurrir que rev(A) = rev(B) (e.g. $A = 123$, $B = 321 \rightarrow$ ambas reversas son 321 y 123 respectivamente — en realidad no). Más: $A = 129$, $B = 921 \rightarrow$ rev(A) = 921, rev(B) = 129, distintas. En la práctica es muy difícil que coincidan dado que los originales son distintos. El enunciado no especifica qué hacer en ese caso; podemos asumir que no ocurre.

Materiales y recursos

- **Cuaderno:** `sesiones/sesion17_taller_1.ipynb`
- **Notas alumno:** `notas_alumno/pdf/sesion17_taller_1.pdf`
- **Problema 2 en omegaUp:** https://www.omegaup.com/arena/problem/La-tarea-de-mi-herm